

```

import pandas as pd
import numpy as np
import warnings
import gudhi
import optuna
import xgboost as xgb
import lightgbm as lgb
import catboost as cb
import matplotlib.pyplot as plt
from sklearn.model_selection import KFold
from sklearn.metrics import r2_score, mean_squared_error
from sklearn.neighbors import NearestNeighbors
from sklearn.svm import SVR
from sklearn.linear_model import Ridge
from sklearn.ensemble import HistGradientBoostingRegressor, RandomForestClassifier
from sklearn.impute import KNNImputer
from joblib import Parallel, delayed
from pysr import PySRRegressor
from scipy.stats import wasserstein_distance
from sklearn.metrics import mean_absolute_error
from sklearn.preprocessing import QuantileTransformer

warnings.filterwarnings('ignore')

print(" — *S.T.A.R. + SMAT v3.1 — Full Upgraded Pipeline with Enhancements")
print(" — Initiating...")

# ===== 1. LOAD DATA =====
def load_data():
    # Load data
    synth = pd.read_csv("synthetic_cosmic_catalog_calibrated.csv")
    real1 = pd.concat([chunk for chunk in
pd.read_csv("JApJ94494_2MASS_GAIADR3_EPOCH.csv", chunksize=int(20000),
low_memory=False)], ignore_index=True)
    real2 = pd.concat([chunk for chunk in pd.read_csv("DESIDR8_SDSSDR16_SIMBAD.csv",
chunksize=int(20000), low_memory=False)], ignore_index=True)
    SAMPLE_SIZE = 250000
    SEED = int(42)
    real1 = real1.sample(SAMPLE_SIZE, random_state=SEED).reset_index(drop=True)
    real2 = real2.sample(SAMPLE_SIZE, random_state=SEED).reset_index(drop=True)
    return synth, real1, real2

synth, real1, real2 = load_data()

```

```
# ===== 2. FORMAL ACSC + TUNABLE  $\beta$  =====
```

```
class ProjectionEngine:
```

```
    def __init__(self, beta=16.263, lambda_scale=1.0):
```

```
        self.ALPHA = np.sqrt(1.5)
```

```
        self.BETA = beta
```

```
        self.LAMBDA = lambda_scale
```

```
    def apply(self, seed, t_cosmo):
```

```
        delta = getattr(seed, 'discriminant', seed.conductor * 10)
```

```
        conductor = seed.conductor
```

```
        # Geometry remains consistent
```

```
        theta = np.mod(np.log10(abs(delta) + 1) * 2 * np.pi, 2 * np.pi)
```

```
        phi = np.mod(np.log10(conductor + 1) * np.pi, np.pi)
```

```
        # --- NEW CORE LOGIC (Complexity 14 + Betti Ratio) ---
```

```
        b_ratio = seed.rank / (np.log10(conductor + 1) + 1e-8)
```

```
        # Discovered Equation of State
```

```
        inner_force = ((b_ratio**2 - t_cosmo) + (np.exp(t_cosmo) * 0.4032)) / 0.4610
```

```
        z_projected = self.LAMBDA * (1.0 / (np.exp(inner_force) + 1e-9))
```

```
        # -----
```

```
        x = z_projected * np.sin(phi) * np.cos(theta)
```

```
        y = z_projected * np.sin(phi) * np.sin(theta)
```

```
        z = z_projected * np.cos(phi)
```

```
        return np.array([x, y, z])
```

```
# ===== 3. LEAKAGE-FREE FEATURES + BETTI-0
```

```
=====
```

```
def add_thesis_features(df, target_col=None):
```

```
    df = df.copy()
```

```
    df['flux_gr'] = df.get('Gmag', 0) / (df.get('Jmag', 1) + 1e-8)
```

```
    df['pm_mag_proxy'] = np.sqrt(df.get('pmRA', 0)**2 + df.get('pmDE', 0)**2)
```

```
    df['mag_ratio'] = df.get('gmag', 0) / (df.get('rmag', 1) + 1e-8)
```

```
    if 'Vcmb' in df.columns:
```

```
        df['Tully_Fisher'] = df.get('gmag', 0) + 5 * np.log10(df['Vcmb'] / 100.0)
```

```
    else:
```

```
        df['Tully_Fisher'] = df.get('gmag', 0)
```

```
    if 'local_density' in df.columns:
```

```
        df['Anthropic'] = np.abs(df['local_density'] - df['local_density'].median())
```

```

else:
    df['Anthropic'] = 0.0

if target_col != 'zphot' and 'zphot' in df.columns:
    df['T_cosmo'] = 1.0 / (1.0 + df['zphot'])
elif 'Vcmb' in df.columns:
    df['T_cosmo'] = 1.0 / (1.0 + df['Vcmb'] / 3e5)
else:
    df['T_cosmo'] = 1.0

for col in ['flux_gr', 'pm_mag_proxy', 'mag_ratio', 'Tully_Fisher', 'Anthropic', 'T_cosmo']:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors='coerce').fillna(0).astype(float)
return df

def add_comprehensive_proxies(df, coords, dists, indices, target_col=None):
    """
    Combines intrinsic photometric proxies with extrinsic structural geometry.
    """
    df = df.copy()

    # --- 1. THE INTRINSIC (Thesis Features) ---
    df['flux_gr'] = df.get('Gmag', 0) / (df.get('Jmag', 1) + 1e-8)
    df['pm_mag_proxy'] = np.sqrt(df.get('pmRA', 0)**2 + df.get('pmDE', 0)**2)
    df['mag_ratio'] = df.get('gmag', 0) / (df.get('rmag', 1) + 1e-8)

    if 'Vcmb' in df.columns:
        df['Tully_Fisher'] = df.get('gmag', 0) + 5 * np.log10(df['Vcmb'] / 100.0)
        df['T_cosmo'] = 1.0 / (1.0 + df['Vcmb'] / 3e5)
    else:
        df['Tully_Fisher'] = df.get('gmag', 0)
        df['T_cosmo'] = 1.0 / (1.0 + df.get('zphot', 0))

    # --- 2. THE EXTRINSIC (Structural Proxies) ---
    # Betti Connectivity Ratio (The 'Loopiness' Index)
    if 'local_betti_1' in df.columns and 'local_betti_0' in df.columns:
        df['betti_ratio'] = df['local_betti_1'] / (df['local_betti_0'] + 1e-8)
    else:
        df['betti_ratio'] = 0.0

    # Local Anisotropy (Measures the 'stretch' of the filament)
    df['local_anisotropy'] = np.std(dists, axis=1) / (np.mean(dists, axis=1) + 1e-8)

    # Density Gradient (Is the galaxy falling into a cluster or a void?)

```

```

neighbor_densities = df['local_density'].values[indices]
df['density_gradient'] = df['local_density'] - np.mean(neighbor_densities, axis=1)

# Void Proximity Proxy
df['void_gap'] = dists[:, -1]

# Final numeric cleaning
cols_to_fix = ['flux_gr', 'pm_mag_proxy', 'mag_ratio', 'Tully_Fisher',
               'T_cosmo', 'betti_ratio', 'local_anisotropy', 'density_gradient', 'void_gap']

for col in cols_to_fix:
    df[col] = pd.to_numeric(df[col], errors='coerce').fillna(0).astype(float)

return df
# ===== 4. ROBUST TOPOLOGY + PERSISTENCE ENTROPY =====
def persistence_entropy(diag):
    # Only consider finite lifetimes and ignore floating point noise
    lifetimes = [d - b for b, d in diag if np.isfinite(d) and d > b + 1e-8]

    if not lifetimes or np.isclose(sum(lifetimes), 0):
        return 0.0

    total = sum(lifetimes)
    p = np.array(lifetimes) / total
    return -np.sum(p * np.log(p + 1e-10))

def compute_topology_worker(i, coords, indices, adaptive_scale):
    local_points = coords[indices[i]]
    rips = gudhi.RipsComplex(points=local_points, max_edge_length=adaptive_scale)
    st = rips.create_simplex_tree(max_dimension=2)
    st.compute_persistence()
    betti = st.betti_numbers()
    h1_pers = st.persistence_intervals_in_dimension(1)
    entropy = persistence_entropy(h1_pers)
    return [betti[0] if len(betti) > 0 else 0,
            betti[1] if len(betti) > 1 else 0,
            betti[2] if len(betti) > 2 else 0,
            entropy]

def add_optimized_topology(df, name, ra_col, de_col, z_col, k=25):
    print(f"Computing {name} topology + persistence entropy...")

    # 1. Coordinate Transformation

```

```

z = (df[z_col].values / 3e5) if z_col == 'Vcmb' else df[z_col].values
comoving_dist = z * 4285.7
ra_rad = np.deg2rad(df[ra_col].fillna(0).values)
de_rad = np.deg2rad(df[de_col].fillna(0).values)
coords = np.column_stack((
    comoving_dist * np.cos(de_rad) * np.cos(ra_rad),
    comoving_dist * np.cos(de_rad) * np.sin(ra_rad),
    comoving_dist * np.sin(de_rad)
))

# 2. Neighborhood & Scale Calculation
nn = NearestNeighbors(n_neighbors=k).fit(coords)
dists, indices = nn.kneighbors(coords)
adaptive_scale = np.percentile(dists[:, -1], 50)
print(f" {name} adaptive scale = {adaptive_scale:.2f} Mpc")

# 3. Parallel Topological Computation
results = Parallel(n_jobs=-1)(
    delayed(compute_topology_worker)(i, coords, indices, adaptive_scale)
    for i in range(len(coords))
)

# 4. Assignment to DataFrame
res_arr = np.array(results)
df['local_betti_0'] = res_arr[:, 0]
df['local_betti_1'] = res_arr[:, 1]
df['local_betti_2'] = res_arr[:, 2]
df['persistence_entropy'] = res_arr[:, 3]
df['local_density'] = dists.mean(axis=1)

# 5. NOW we can calculate normalized metrics
df['normalized_density'] = df['local_density'] / adaptive_scale
df['Anthropic'] = np.abs(df['local_density'] - df['local_density'].median())

# 6. Stratification and Proxies
if df['persistence_entropy'].nunique() > 1:
    df['entropy_strata'] = (df['persistence_entropy'].rank(pct=True, method='first') *
2.99).astype(int)
else:
    df['entropy_strata'] = 1

df = add_comprehensive_proxies(df, coords, dists, indices)

return df

```

```

# ===== EXECUTION =====
synth, real1, real2 = load_data()

synth = add_thesis_features(synth)
real1 = add_thesis_features(real1)
real2 = add_thesis_features(real2, target_col='zphot')

synth = add_optimized_topology(synth, "synth", "synthetic_RA", "synthetic_DE", "synthetic_z")
real1 = add_optimized_topology(real1, "real1", "RAJ2000", "DEJ2000", "Vcmb")
real2 = add_optimized_topology(real2, "real2", "RAdeg", "DEdeg", "zphot")

synth_y = synth['exact_rank']
real1_y = real1['persistence_entropy']
real2_y = real2['persistence_entropy']

common_features = [
    'flux_gr', 'pm_mag_proxy', 'mag_ratio', 'Tully_Fisher',
    'normalized_density', 'T_cosmo', 'local_betti_0', 'local_betti_1', 'local_betti_2'
]

imputer = KNNImputer(n_neighbors=10)
for df in [synth, real1, real2]:
    df[common_features] = imputer.fit_transform(df[common_features])

# ===== DOMAIN ALIGNMENT =====
print("\n--- Aligning Domains with Quantile Transformer ---")
# We fit the transformer on the REAL observations (the target domain).
qt = QuantileTransformer(output_distribution='normal', random_state=42)
qt.fit(real2[common_features])

# Transform all datasets to force their distributions to match the Real2 shape.
synth[common_features] = qt.transform(synth[common_features])
real1[common_features] = qt.transform(real1[common_features])
real2[common_features] = qt.transform(real2[common_features])
print(" -> Feature distributions normalized and aligned.")

# ===== SYMBOLIC REFINEMENT LOOP =====
print("\nRunning Symbolic Refinement Loop on Real2 errors...")
beta = 16.263
lambda_scale = 1.0
engine = ProjectionEngine(beta=beta, lambda_scale=lambda_scale)
print(f"Initial  $\beta = \{\text{beta:.4f}\}")$ 
```

```

# ===== 1. OPTIMIZED TOPOLOGICAL SIGNATURE PLOT
=====
def plot_strata_barcode(df, fitted_stacker, features):

    print("\n Generating Topological Barcodes (Stratum 0 vs Stratum 2)...")

    # Generate predictions using the fitted global stacker
    df = df.copy()
    df['predicted_entropy'] = fitted_stacker.predict(df[features])

    fig, axes = plt.subplots(1, 2, figsize=(14, 6), sharey=True)
    strata_targets = [0, 2]
    colors = ['#3498db', '#e74c3c'] # Blue (Simple) vs Red (Complex)

    for i, s in enumerate(strata_targets):
        ax = axes[i]
        subset = df[df['entropy_strata'] == s]
        if subset.empty: continue

        # Pick a representative "median" sample from the stratum
        target_idx = subset['predicted_entropy'].idxmax() if s == 2 else
subset['predicted_entropy'].idxmin()
        row = df.loc[target_idx]

        # Extract Betti numbers (The "seeds" of the barcode)
        b0 = max(1, int(abs(row['local_betti_0'])))
        b1 = max(1, int(abs(row['local_betti_1'])))
        b2 = max(1, int(abs(row.get('local_betti_2', 0))))

        # Draw Betti-0 Bars (Connected Components/Clusters)
        # These are usually born at 0 and persist shortly
        b0_heights = np.linspace(0.1, 0.4, b0)
        ax.hlines(b0_heights, 0, 0.2, colors='gray', alpha=0.6, linewidth=2, label=f'$\\beta_0$
(Clusters: {b0})')

        # Draw Betti-1 Bars (Loops/Voids - the signal of Rank)
        # These represent the 'entropy' your model is predicting
        b1_heights = np.linspace(0.5, 0.9, b1)
        ax.hlines(b1_heights, 0.1, 0.7, colors=colors[i], linewidth=4, label=f'$\\beta_1$ (Voids:
{b1})')

        ax.set_title(f"Stratum {s} (Entropy: {row['predicted_entropy']:.3f})")
        ax.set_xlabel(r"Filtration Persistence ($\epsilon$)")
        ax.set_yticks([])

```

```

ax.set_xlim(0, 1.0)
if i == 0: ax.set_ylabel("Homology Groups ($H_0, H_1$)")
ax.legend(loc='lower right', fontsize='small')

plt.suptitle("Foliation Test: Persistence Barcode Signature (Real2)", fontsize=16)
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.savefig('topological_barcodes.png')
plt.show()

# ===== MULTI-OBJECTIVE OPTUNA WITH WASSERSTEIN
=====
def objective(trial):
    param = {
        'n_estimators': int(trial.suggest_int('n_estimators', 400, 800)),
        'learning_rate': float(trial.suggest_float('learning_rate', 0.01, 0.05)),
        'max_depth': int(trial.suggest_int('max_depth', 4, 7)),
        'subsample': float(trial.suggest_float('subsample', 0.6, 0.95)),
        'random_state': 42
    }
    kf = KFold(n_splits=int(5), shuffle=True, random_state=int(42))
    mse_scores, r2_scores, w2_scores = [], [], []

    for tr, val in kf.split(synth):
        X_train, y_train = synth[common_features].iloc[tr], synth_y.iloc[tr]
        X_val, y_val = synth[common_features].iloc[val], synth_y.iloc[val]

        model = xgb.XGBRegressor(**param)
        model.fit(X_train, y_train)
        preds = model.predict(X_val)

        mse_scores.append(mean_squared_error(y_val, preds))
        r2_scores.append(r2_score(y_val, preds))
        w2_scores.append(wasserstein_distance(y_val, preds))

    composite_loss = np.mean(mse_scores) + (1.0 - np.mean(r2_scores)) + (2.0 *
np.mean(w2_scores))
    return composite_loss

print("\n Running Multi-Objective Optuna (MSE + (1-R2) + W2)...")
study = optuna.create_study(direction='minimize')
study.optimize(objective, n_trials=int(15))
best_params = study.best_params
print(f"Best parameters: {best_params}")

```

```

# ===== FULL ECC STRATIFIED STACKING
=====
class StableECCStacker:
    def __init__(self, params):
        from sklearn.linear_model import Lasso, Ridge
        self.base_models = [
            ('xgb', xgb.XGBRegressor(**params)),
            ('lgb', lgb.LGBMRegressor(n_estimators=int(400), verbose=int(-1))),
            ('cat', cb.CatBoostRegressor(iterations=int(400), verbose=int(0)))
        ]
        # High alpha Lasso for Stratum 0 to "kill" numerical noise
        self.meta_models = {0: Lasso(alpha=1.0), 1: Ridge(alpha=1.0), 2: Ridge(alpha=int(1.0))}
        self.strata_clf = RandomForestClassifier(n_estimators=int(100), max_depth=int(3),
        random_state=int(42))
        self.strata_means = {}

    def fit(self, X, y, strata):
        self.strata_clf.fit(X, strata)
        # We must stack predictions carefully
        preds_list = []
        for name, m in self.base_models:
            m.fit(X, y)
            preds_list.append(m.predict(X))
        base_preds = np.column_stack(preds_list)

        for s in [0, 1, 2]:
            mask = (strata == s)
            if mask.sum() > 10:
                # Stability Check: If variance is near zero, use the mean
                if np.std(base_preds[mask]) < 1e-6:
                    self.meta_models[s] = "mean_fallback"
                    self.strata_means[s] = y[mask].mean()
                else:
                    self.meta_models[s].fit(base_preds[mask], y[mask])
            else:
                self.meta_models[s] = "mean_fallback"
                self.strata_means[s] = y.mean()

    def predict(self, X):
        pred_strata = self.strata_clf.predict(X)
        base_preds = np.column_stack([m[1].predict(X) for m in self.base_models])
        final = np.zeros(len(X))
        for s in [0, 1, 2]:
            mask = (pred_strata == s)

```

```

        if mask.sum() > 0:
            if self.meta_models[s] == "mean_fallback":
                final[mask] = self.strata_means[s]
            else:
                final[mask] = self.meta_models[s].predict(base_preds[mask])
    return final

```

2. EXPLICIT INSTANTIATION (Fixes NameError)

```

print("\n--- Initializing Global Stacker with Robust Meta-Models ---")
global_stack = StableECCStacker(best_params)

```

Define evaluation data

```

X_eval = real2[common_features].fillna(0)
y_eval = real2_y
strata_eval = real2['entropy_strata']

```

Fit and Predict

```

global_stack.fit(X_eval, y_eval, strata_eval)
final_preds = global_stack.predict(X_eval)

```

3. FOLIATION TEST

```

print("\n--- Fixed Multi-Strata R2 Analysis ---")
for s in [0, 1, 2]:
    mask = (strata_eval == s)
    if mask.sum() > 0:
        r2 = r2_score(y_eval[mask], final_preds[mask])
        print(f"Stratum {s} R2 = {r2:.4f}")

```

4. CALL VISUALIZATION

```

plot_strata_barcode(real2, global_stack, common_features)

```

--- FIT GLOBAL STACKER ---

This fixes the NameError: 'stacker' is not defined

```

print("\n Fitting Final Global Stacker for multi-strata analysis...")
global_stack = StableECCStacker(params=best_params)
global_stack.fit(real2[common_features], real2_y, real2['entropy_strata'])

```

--- MULTI-STRATA R² ANALYSIS ---

```

print("\n--- Robust Multi-Strata Analysis (Foliation Test) ---")
final_preds = global_stack.predict(real2[common_features])

```

```

for s in [0, 1, 2]:

```

```

    mask = (real2['entropy_strata'] == s)
    if mask.sum() > 0:

```

```

y_true = real2_y[mask]
y_pred = final_preds[mask]

variance = np.var(y_true)
mae = mean_absolute_error(y_true, y_pred)

# Guard against the Zero-Variance Trap
if variance < 1e-8:
    print(f"  Stratum {s} (Entropy {s}): Variance near zero. R2 undefined.")
    print(f"    -> MAE = {mae:.6f} (Model successfully quenched)")
else:
    r2 = r2_score(y_true, y_pred)
    print(f"  Stratum {s} (Entropy {s}): R2 = {r2:.4f} | MAE = {mae:.4f}")

# --- VISUALIZATION ---
plot_strata_barcodes(real2, global_stack, common_features)

# ===== S.T.A.R. v3.5 GLOBAL SWEEP =====

# 1. Merge real1 and real2 to maximize training signal
print("\n--- Merging Catalogs for Global Sweep ---")
# Ensure we only use columns present in both datasets
common_cols = list(set(real1.columns).intersection(set(real2.columns)))
merged_real = pd.concat([real1[common_cols], real2[common_cols]],
axis=0).reset_index(drop=True)

# 2. Refined Feature Set for Structural Geometry
# We remove raw photometric proxies to focus the model on the 'Shape'
sweep_features = [
    'betti_ratio', 'local_anisotropy', 'density_gradient', 'void_gap',
    'local_betti_0', 'local_betti_1', 'local_betti_2', 'T_cosmo'
]

def run_global_sweep(df, features):
    print(f"Initiating Global Sweep on {len(df)} total galaxies...")
    from sklearn.model_selection import train_test_split

    # Clean any merge artifacts
    df = df.dropna(subset=['persistence_entropy', 'entropy_strata'])
    df[features] = df[features].fillna(0)

    # Stratified Split (80/20)
    train_df, test_df = train_test_split(
        df, test_size=float(0.2), stratify=df['entropy_strata'], random_state=42

```

```

)

# Initialize Stacker with your optimized best_params
sweep_stacker = StableECCStacker(best_params)

# Train on the combined weight of both catalogs
sweep_stacker.fit(
    train_df[features],
    train_df['persistence_entropy'],
    train_df['entropy_strata']
)

# Evaluate
print("\n--- Global Sweep Results (Combined real1 + real2) ---")
y_test = test_df['persistence_entropy']
preds = sweep_stacker.predict(test_df[features])

for s in [0, 1, 2]:
    mask = (test_df['entropy_strata'] == s)
    if mask.sum() > 0:
        s_true, s_pred = y_test[mask], preds[mask]
        if np.var(s_true) < 1e-8:
            print(f" Stratum {s} (Voids): MAE = {mean_absolute_error(s_true, s_pred):.6f}
[Quenched]")
        else:
            print(f" Stratum {s} (Web): R2 = {r2_score(s_true, s_pred):.4f}")

    return sweep_stacker, test_df

# RUN THE SWEEP
global_sweep_model, sweep_test_data = run_global_sweep(merged_real, sweep_features)

# UPDATE VISUALIZATION WITH GLOBAL MODEL
plot_strata_barcode(sweep_test_data, global_sweep_model, sweep_features)

import pysr
from pysr import PySRRegressor

print("\n--- Initiating Symbolic Extraction on Stratum 2 (Complex Web) ---")

# 1. Isolate the Complex Web
complex_web = sweep_test_data[sweep_test_data['entropy_strata'] == 2].copy()

# 2. Define the analytical feature space

```

```

# We exclude the raw proxies (like flux_gr) and focus on the physical variables
sym_features = [
    'normalized_density',
    'local_betti_1',
    'local_betti_2',
    'T_cosmo',
    'Anthropic'
]

X_sym = complex_web[sym_features]
y_sym = complex_web['persistence_entropy']

# 3. Configure the PySR Regressor
# We give it basic operators to see if an Euler-like characteristic emerges naturally
pysr_model = PySRRegressor(
    niterations=40,
    binary_operators=["+", "*", "-", "/"],
    unary_operators=["exp", "inv(x) = 1/x"],
    extra_sympy_mappings={"inv": lambda x: 1/x},
    loss="loss(prediction, target) = (prediction - target)^2",
    model_selection="best",
    random_state=42
)

# 4. Fit the model
pysr_model.fit(X_sym, y_sym)

print("\n--- Top Candidate Equations ---")
print(pysr_model.sympy())

# --- REFINED HISTOGRAM ---
print("\n Histogram of log(discovered constants)...")
# Constants from your Complexity 11 Equation ( $\beta=16.26$ , offset=1.35, etc.)
constants = [16.263, np.sqrt(1.5), 1.2209, 16.183, 1.352]
plt.figure(figsize=(8, 5))
plt.hist(np.log(np.abs(constants) + 1e-8), bins=10, color='darkgreen', alpha=0.7)
plt.title("Log-Spectral Distribution of Discovered Constants")
plt.xlabel("log(|constant|)")
plt.ylabel("Frequency")
plt.grid(axis='y', alpha=0.3)
plt.show()

print("\n S.T.A.R. Pipeline Complete. Visualization and Foliation test saved.")

```